DE10 Audio Game

SWE-350
11/9/2025

Nolan Tuttle

Grand Canyon University

College of Engineering and Technology

Software Engineering Program

Embedded Systems 1

Table of Contents

Project Demonstration Link.....2
Github Repo Link.....2
System Design.....3
Application Logic Design.....7
Risk Matrix.....11
Test Plan.....12
Known Issues.....13
Misc Documentation.....14

Project Demonstration Link

https://www.youtube.com/watch?v=SPP4C422jtY

GitHub Repo Link

https://github.com/nolantuttle/DE10AudioGame

System Design

General Technical Approach:

This project is an audio-based memory game running on Intel's SoC FPGA platform. Running on the DE10 Standard embedded board, it maps the four buttons to four unique audio samples and generates a sequence of these audio samples to play from the WM8731 audio codec through the line out port on the board. The goal of the player is to press the buttons which are mapped to the four audio samples to generate the same sequence of audio samples output by the board, with each round the sequence will increment by 1, increasing difficulty of the game. By utilizing both the Hard Processing System (HPS) and FPGA fabric, the application delivers a responsive interaction that combines audio, button input, and hex display feedback.

The HPS is responsible for executing the game logic features of this application. For example, the audio sample sequence generation/randomization as well as keeping track of sequence length. The HPS will check to see if the user's button input is correct and matches the sequence, if not it is also responsible for resetting the game upon failure. The HPS communicates the audio samples to play over the lightweight AXI bridge into the FPGA's memory mapped I/O.

Audio output is configured using an I2C line connected to the FPGA and HPS, although the HPS only is responsible for the actual configuration. The output is set to operate in 16-bit I2S mode at 48 kHz so that it is prepared for audio input from the FPGA over I2S. The audio codec functions in slave mode, allowing the clock cycles to be set by the FPGA through its VLSI logic. Game progress and feedback such as current sequence length is displayed on the hex display. Currently the I2C configuration sets the audio into DSP mode, accepting on-chip memory storage as the audio sample.

Development of HPS logic such as C files was performed using Visual Studio Code, while compilation, linking, and running occurs through a Makefile-based build system. The structure of the project is modular and separated into sections in the filesystem as shown in Figure 10.

System Design

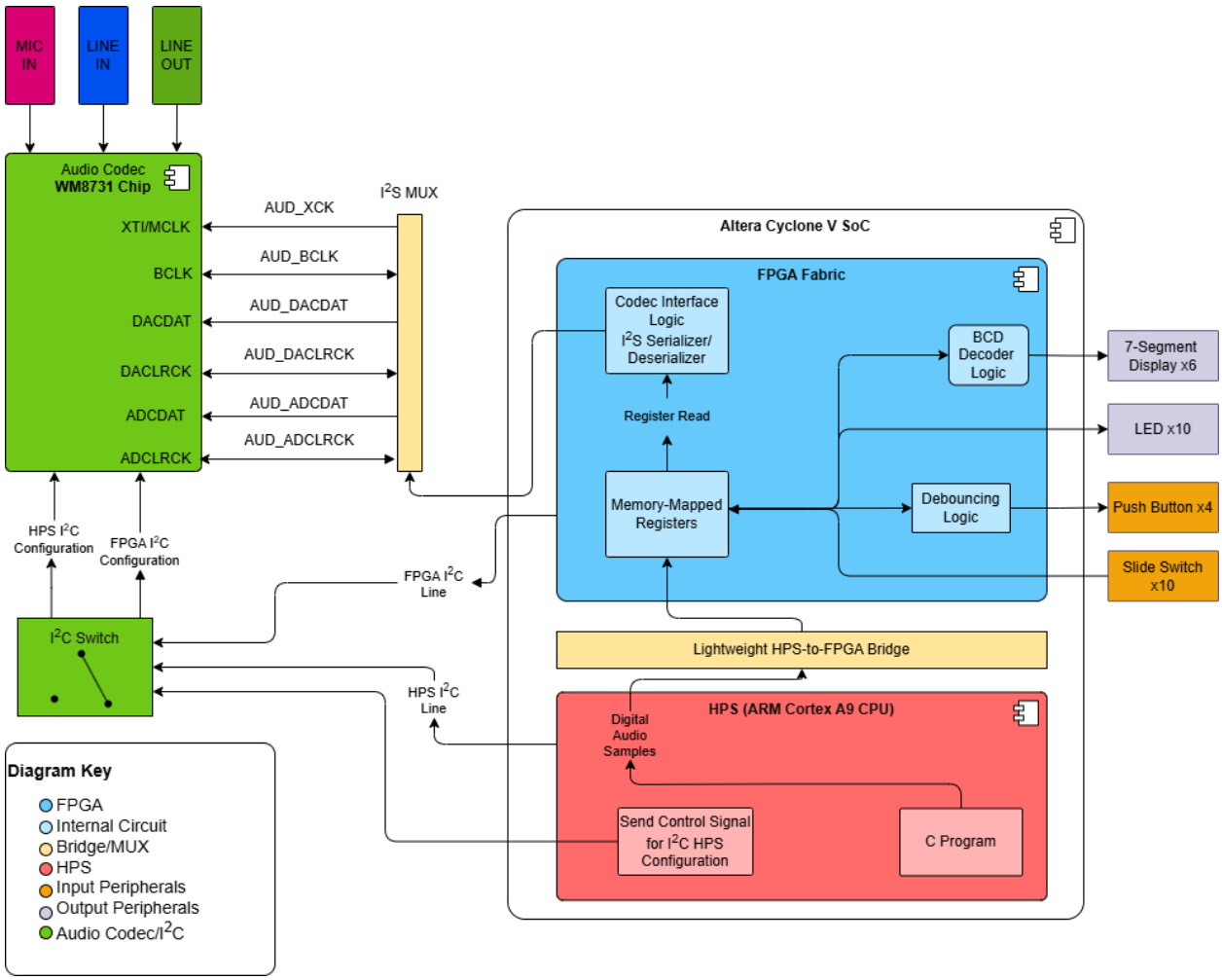

Figure 1: *System Design Component Diagram for audio output*

Application Logic Design

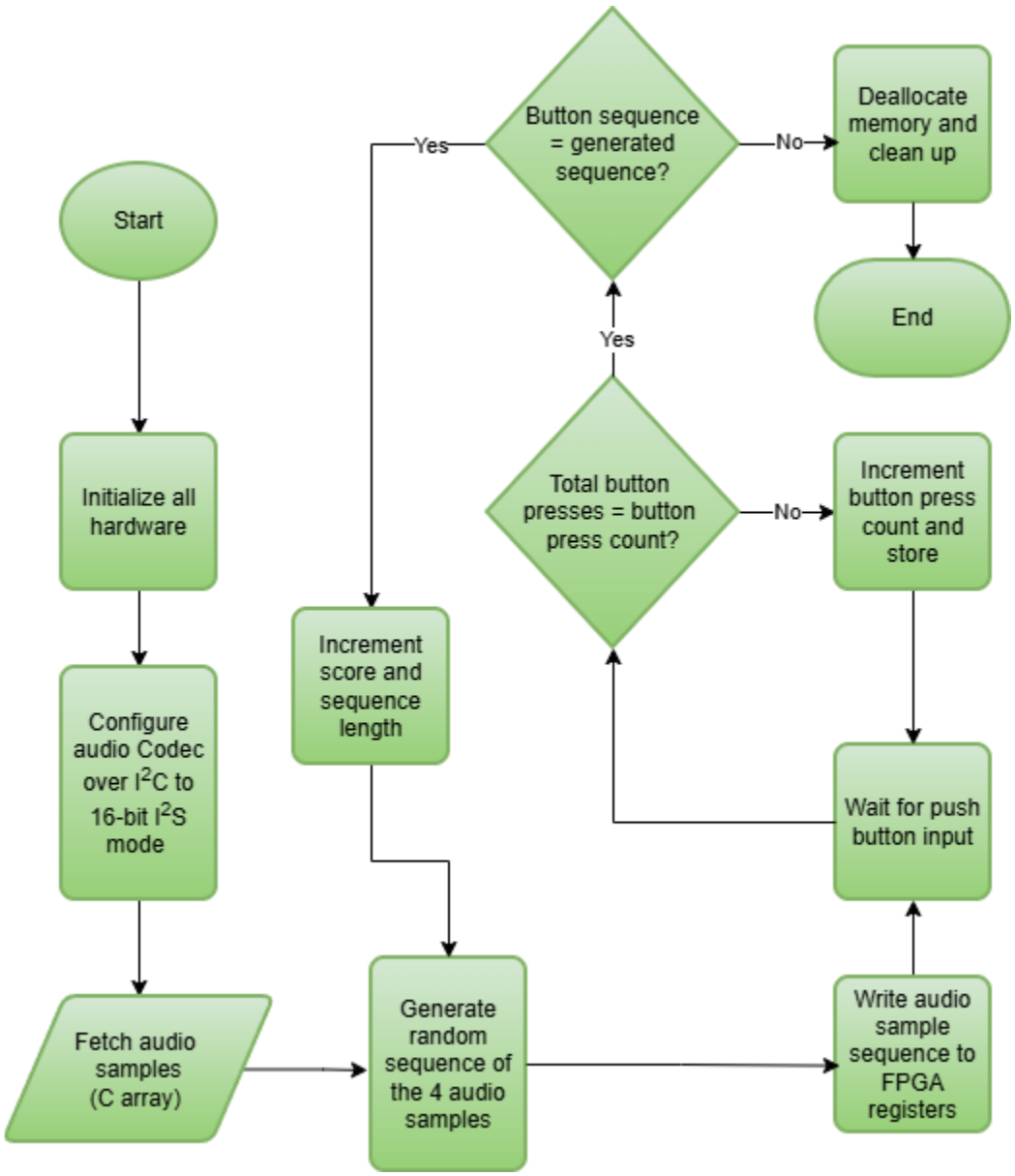

Figure 2: *Application Logic Design for HPS*

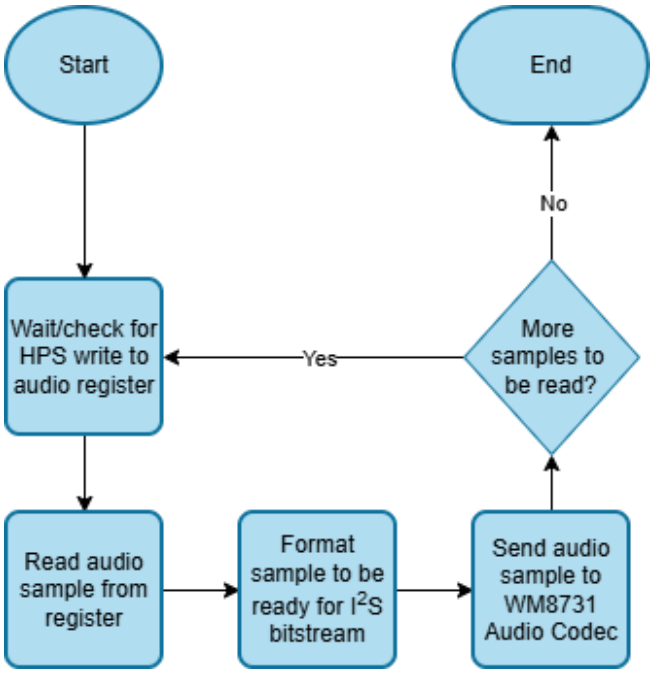

Figure 3: *Application Logic Design for FPGA*

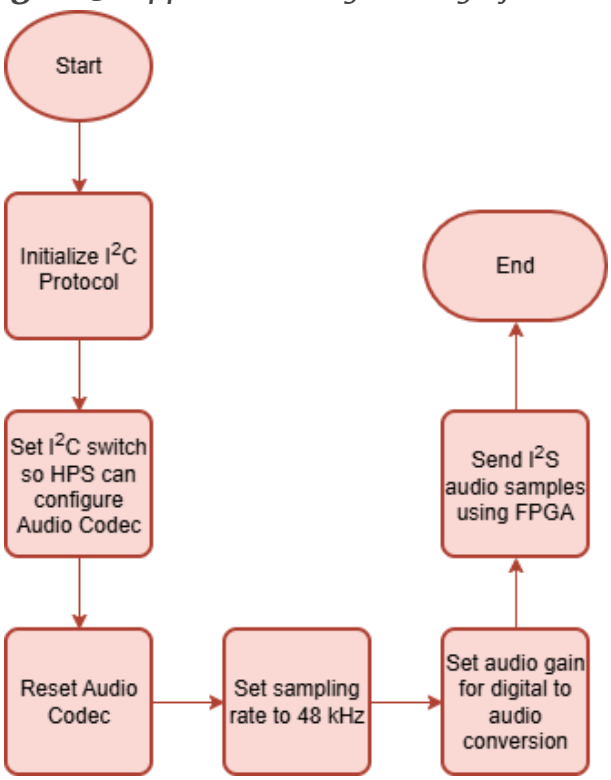

Figure 4: *Application Logic Design for Audio Codec*

```
START
// Initialize all hardware
Init_hardware()
Configure_codec(bit_depth, sampling_rate, gain)

// Game tracking variables
Sequence = []
Pattern_length = 1
Game_over = FALSE

      WHILE !(Game_over)

            New_sample = generate_sample()
            Sequence.append(new_sample)

                  FOR sample IN Sequence
                        Sample_to_FPGA(sample)
                        Wait_for_playback()
                  END FOR
                  // User input
                  FOR num IN pattern_length
                        Input = read_button()
                        If(Input != sequence[num])
                              Game_over = TRUE
                              BREAK
                        END IF
                  END FOR

                  IF NOT Game_over
                        Pattern_length+=1
                  END IF
      END WHILE

      Game_complete_display()
END
```

Figure 5: *Main Application Pseudocode Logic*

VLSI Design

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;


entity  audio_codec is
port (


----------WM8731 pins-----
AUD_BCLK: out std_logic;
AUD_XCK: out std_logic;
AUD_ADCLRCK:  out std_logic;
AUD_ADCDAT: in std_logic;
AUD_DACLRCK: out std_logic;
AUD_DACDAT: out std_logic;

---------FPGA pins-----

clock_50: in std_logic;
key: in std_logic_vector(3 downto 0);
ledr: out std_logic_vector(9 downto 0);
sw: in std_logic_vector(9 downto 0);
FPGA_I2C_SCLK: out std_logic;
FPGA_I2C_SDAT: inout std_logic

);

end audio_codec;
```

Figure 6: *Audio Codec VHDL Example (Toni, 2017)*

```
begin

aud_bk<=aud_clock_12;

process(aud_clock_12)
begin

if falling_edge(aud_clock_12) then

      aud_dalr<=clk_en;

      if(aud_prscl<250)then------48k sample rate
            aud_prscl<=aud_prscl+1;
            clk_en<='0';
            else
            aud_prscl<=0;
            da_data_out<=aud_data_in;--get sample
            clk_en<='1';
      end if;



      if(clk_en='1')then-------send new sample
      sample_flag<='1';
      data_index<=31;
      end if;

      if(sample_flag='1')then

            if(data_index>0)then
                  aud_dadat<=da_data_out(data_index);
                  data_index<=data_index-1;
                  else
                  aud_dadat<=da_data_out(data_index);
                  sample_flag<='0';
            end if;

      end if;
end if;
```

Figure 7: *Audio Generation Example VHDL (Toni, 2017)*

Risk Matrix

ID	Category	Description	Severity	Mitigation
1	Scope	The audio I2S pipeline from the C program on the HPS to the audio codec is highly complex, so there is a risk of faulty system design resulting in functional issues or system failure.	H	**Reduction** - Mirror existing official documentation from Terasic, conduct extensive research on communication protocols like I2S and I2C, as well as the WM8731 audio codec.
2	Technical	Because setting the clock rate for the codec is done using the clock division method (shown in Figure 7), the FPGA clock division might not match the sample rate needed for the codec.	M	**Reduction** - Adjust the FPGA clock division in the FPGA Verilog logic.
3	Technical	When configuring the audio codec over I2C using the HPS, an incorrect configuration (wrong sampling rate, gain, etc.) (see Figure 4), can cause prevented audio output.	M	**Reduction** - Validate the correct register writes happen over I2C, test with a known audio codec configuration.
4	Technical	The FPGA must be configured in such a way that the HPS only sends data when it is ready to receive data in its registers. There is a risk that these are not in sync with each other and data is sent when FPGA is not ready.	H	**Avoidance** - Use a handshaking method between HPS and FPGA, something like the flag hps_data_ready in the Verilog logic (Figure 6).
5	Operational	Latency in audio playback or peripheral responsiveness	L	**Acceptance** -Replace more HPS logical operations with

		may hinder UX when running the game.		FPGA programming to improve latency.

Test Plan

#	Title	Summary	Steps	Data	Expected Result	Status
1	FPGA Button IRQ interaction	Verify that the interrupt request module for the pushbutton works properly	1. Power on board 2. Load the kernel module (irq_pushbutton.ko) 3. Push any button 4. Verify LED feedback	The analog data from the button press being passed to the HPS.	The LEDs should increment their binary count representation with each consecutive button press.	**Pass**

Known Issues

ID	Description	Plan of Address
1	N/A	N/A

Misc Documentation

One important aspect to understand about the I2S communication between the FPGA and the audio codec chip in this project is that, unlike a system where the codec chip works as a master, determining the necessary clock cycles for itself, it is better in this application for the audio codec chip to be in slave configuration, letting the FPGA generate the necessary clocks. The two clocks here are the serial clock (sck) and word select (ws) pictured below in Figure 8 and as used in the Verilog logic in Figure 7. Figure 8 shows a block diagram for what the shift register that receives the audio data might look like with a master codec design. Note that the below diagram is a master configuration, not slave configuration.

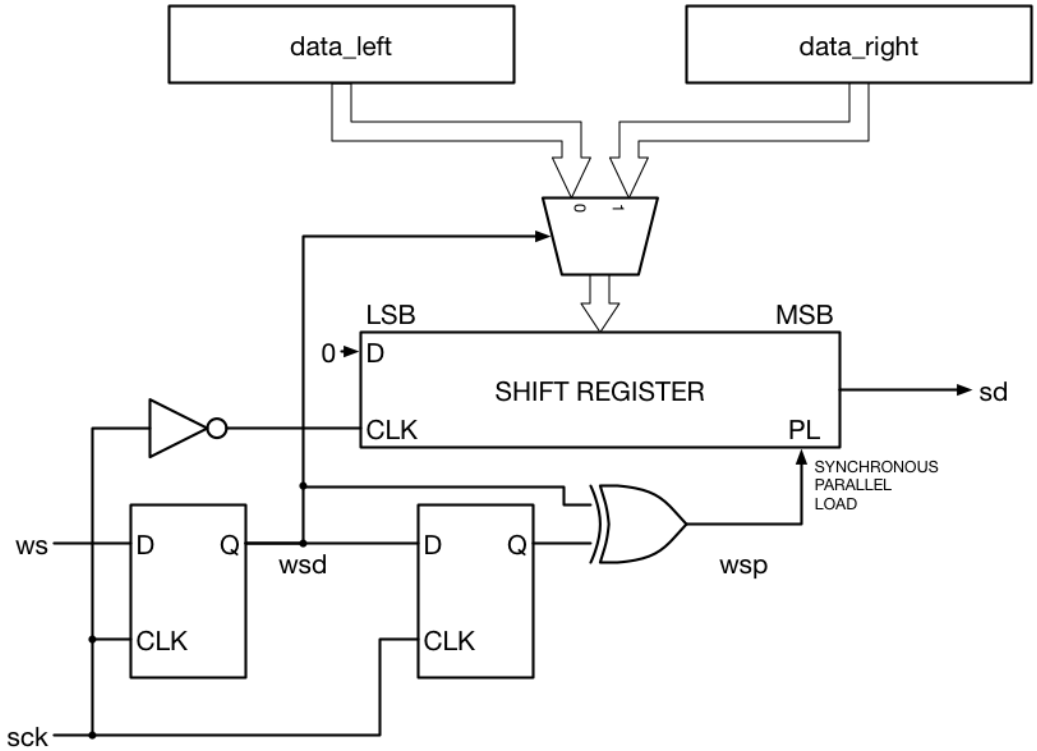

Figure 8: Traditional I2S transmitter block diagram. Reprinted from Beyond Circuits, "Tutorial 19: I2S Transmitter," https://www.beyond-circuits.com/wordpress/tutorial/tutorial19/.

Using a master configuration as shown above in this embedded application could result in inaccurate audio samples because there is a risk of generating inaccurate frequencies since it is handled by the codec. The bigger risk here, however, is synchronizing the FPGA clock with the WM8731 Codec clock. If the FPGA handles the clock generation, such as in a slave design, the FPGA has full control over timing, shown by the FPGA clock S_AXIS_ACLK shown in Figure 9. This makes it far easier to synchronize HPS audio sample writes to FPGA registers that will output audio; however, the FPGA must generate accurate clocks. This is an important distinction because it affects the Verilog significantly; the FPGA must generate clocks and thus, requires modules that can help the system function in slave mode.

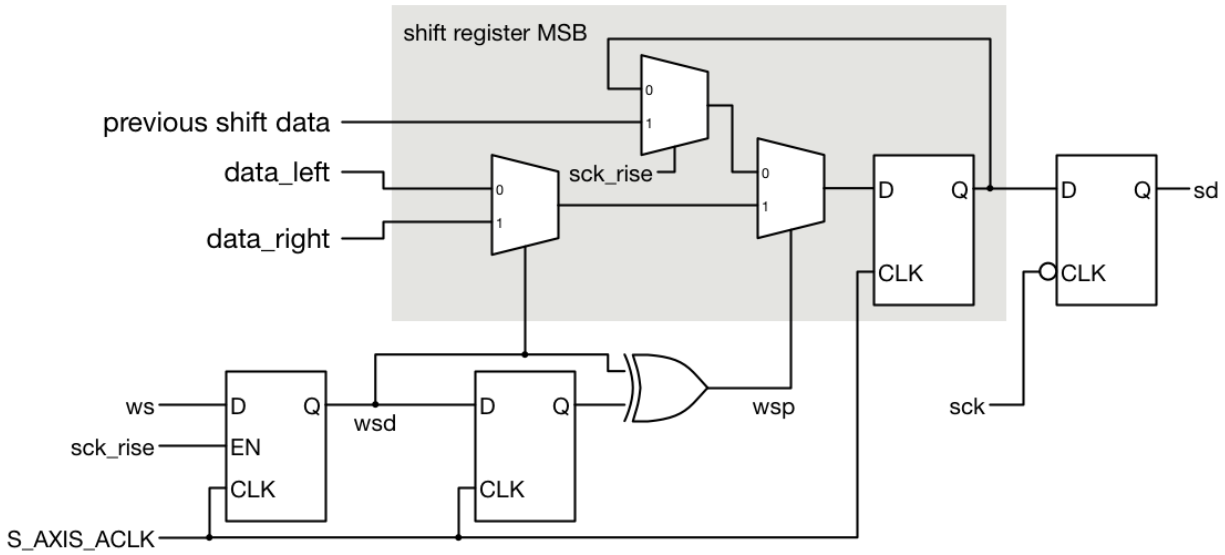

Figure 9: FPGA Clock I2S transmitter block diagram. Reprinted from Beyond Circuits, "Tutorial 19: I2S Transmitter," https://www.beyond-circuits.com/wordpress/tutorial/tutorial19/.

```
.
└── audioGame/
    ├── src/
    │   ├── main.c
    │   ├── hps_audio.c
    │   └── game_logic.c
    ├── include/
    │   ├── hal_api.h
    │   ├── hex_display.h
    │   ├── hps_audio.h
    │   ├── buttons.h
    │   └── game_logic.h
    ├── lib/
    │   └── address-map-arm.h
    ├── test/
    │   ├── test_audio.c
    │   └── test_buttons.c
    ├── bin/
    │   └── audioGame
    ├── obj/
    │   ├── buttons.o
    │   ├── game_logic.o
    │   ├── hal_api.o
    │   ├── hex-display.o
    │   ├── hps_audio.o
    │   └── main.o
    └── Makefile
```

Figure 10: File Tree Structure for Audio Game project

Reference

Beyond Circuits. (n.d.). Tutorial 19: I2S Transmitter. Retrieved October 18, 2025, from https://www.beyond-circuits.com/wordpress/tutorial/tutorial19/

Toni. (2017, May 16). Tutorial: Playing audio through WM8731 using Terasic DE10-Standard FPGA development board [Video]. YouTube. https://www.youtube.com/watch?v=zzIi7ErWhAA